

3. Programació amb arrays i objectes

En aquesta lliçó explorarem com JavaScript permet treballar amb dades estructurades i comportaments associats mitjançant objectes. Aprendre a crear i gestionar **arrays** i **objectes** definits pel programador, i introduïrem els principis bàsics de la programació orientada a objectes (POO), com ara classes, propietats i mètodes. Finalment, veurem alguns exemples pràctics que mostren com aplicar aquests conceptes per escriure codi més modular, reutilitzable i fàcil de mantenir.

3.1 Arrays. Objectes definits pel programador

En aquest apartat veurem com JavaScript ens permet organitzar i gestionar dades de manera eficient mitjançant arrays i objectes. Els arrays són col·leccions ordenades d'elements, ideals per emmagatzemar llistes o conjunts de valors, mentre que els objectes ens permeten agrupar informació relacionada en una mateixa estructura amb claus i valors. A més, combinant aquests elements, podem crear estructures de dades complexes adaptades a les necessitats del nostre programa.

3.1.1 Arrays com estructures de dades

Un **array** és una estructura que ens permet **emmagatzemar diversos valors en una sola variable**, de forma **ordenada**. És molt útil quan volem treballar amb **llistes de dades**, com noms, números, objectes, etc.

Els arrays en JavaScript poden contenir **qualsevol mena de dada**, fins i tot barrejats (nombres, *strings*, booleans, objectes, altres arrays...).

Creació d'un array

A continuació, veurem alguns exemples de com crear arrays.

```
1 let buida = []; // Array buit
2 let numeros = [1, 2, 3, 4]; // Array numèric
3 let noms = ["Anna", "Pau", "Joan"]; // Array d'Strings
4 let mesclat = [1, "hola", true, [10, 20]]; // Array mixt
```

També es pot fer amb el constructor:

```
1 let colors = new Array("vermell", "blau", "verd");
```

Tot i això, la forma recomanada és la de claudàtors ([]), per ser més clara i segura.

Accés als elements

Els elements es compten des de l'índex zero (0 és el primer element).

```
1 let numeros = [10, 20, 30];
2 console.log(numeros[0]); // 10
3 console.log(numeros[2]); // 30
```

Longitud de l'array

Cada array té una propietat `.length` que indica quants elements conté:

```
1 numeros.length; // 3
```

Aquesta propietat també es pot usar per recórrer l'array amb un bucle:

```
1 for (let i = 0; i < numeros.length; i++) {
2   console.log(numeros[i]);
3 }
```

Mètodes útils dels arrays

Els arrays disposen de molts **mètodes integrats** per facilitar les operacions habituals. A continuació veurem un llistat dels més usuals:

TAULA 3.1. Mètodes útils dels arrays

Mètode	Què fa
<code>push()</code>	Afegeix un element al final
<code>pop()</code>	Elimina i retorna l'últim element
<code>unshift()</code>	Afegeix un element al principi
<code>shift()</code>	Elimina i retorna el primer element
<code>join(separador)</code>	Uneix tots els elements en un <i>string</i> , separats per separador. El separador per defecte és la coma ,
<code>concat()</code>	Uneix dos arrays en un de sol
<code>slice(inici, fi)</code>	Retorna una part de l'array (no modifica l'original)
<code>splice()</code>	Afegeix/elimina elements de l'array (sí que modifica l'original)
<code>indexOf(elem)</code>	Retorna la posició d'un element (o -1 si no hi és)
<code>includes(elem)</code>	Indica si un element es troba dins l'array

Anem a veure alguns exemples del mètodes anteriors:

```
1 let fruites = ["poma", "plàtan", "kiwi"];
2
3 fruites.push("maduixa");           // ["poma", "plàtan", "kiwi", "
   maduixa"]
```

```
4 fruites.shift();           // ["plàtan", "kiwi", "maduixa"]
5 fruites.includes("kiwi");   // true
6 fruites.indexOf("plàtan");  // 0 (Ha canviat després d'
    aplicar el mètode shift)
```

Iterar amb ".forEach()"

El mètode `.forEach()` ens permet **recórrer tots els elements d'un array** i executar una funció per a cadascun d'ells. És una alternativa moderna i més llegible que els bucles `for`.

No retorna cap nou array: simplement **executa una acció per cada element**.

```
1 let animals = ["gat", "gos", "ocell"];
2
3 animals.forEach(function(animal, index) {
4     console.log(index + ": " + animal);
5 });
```

També es pot fer amb funcions fletxa:

```
1 animals.forEach((animal, i) => console.log(i + ": " + animal));
```

Transformar arrays: ".map()" i ".filter()"

Els mètodes `.map()` i `.filter()` ens permeten crear un nou array a partir de l'original:

- `map()` transforma cada element i retorna un array nou amb els resultats.
- `filter()` selecciona només els elements que compleixen una condició.

Aquests mètodes no modifiquen l'array original.

```
1 let nums = [1, 2, 3, 4];
2
3 // Multiplica per 2 cada element
4 let dobles = nums.map(n => n * 2); // [2, 4, 6, 8]
5
6 // Filtra els nombres parells
7 let parells = nums.filter(n => n % 2 === 0); // [2, 4]
```

3.1.2 Objectes com estructures de dades

A JavaScript, un **objecte** és una estructura que ens permet **emmagatzemar dades en forma de clau-valor**. És una eina fonamental per representar entitats més complexes i flexibles que un array, com ara una persona, un cotxe, un usuari, etc.

Mentre que els arrays organitzen les dades per **posició numèrica (índexs)**, els objectes ho fan per **claus amb nom (strings)**.

Creació d'un objecte

A continuació, es mostra com crear un **objecte literal**. Un objecte literal en JavaScript és una forma directa i senzilla de crear un objecte amb *claus* (propietats) i *valors*.

```
1 let persona = {  
2   nom: "Anna",  
3   edat: 28,  
4   viu: true  
5 };
```

Cada propietat té una *clau* (ex. “nom”) i un *valor* associat (ex. “Anna”).

Accés a propietats

Podem accedir a les propietats amb notació de punt o notació de claudàtors:

```
1 // Punt (dot notation)  
2 console.log(persona.nom); // "Anna"  
3  
4 // Claudàtors (bracket notation)  
5 console.log(persona["edat"]); // 28
```

Notació amb claudàtors

La notació amb claudàtors és útil quan la clau és una variable o conté espais:

```
1 let clau = "viu";  
2 console.log(persona[clau]); // true
```

Modificar i afegir propietats

Els objectes en JavaScript són **dinàmics**, la qual cosa vol dir que podem afegir noves propietats o modificar-ne les existents en qualsevol moment, fins i tot després d’haver estat creats.

Aquesta flexibilitat fa que els objectes siguin molt útils com a contenidors d’informació estructurada.

```
1 persona.edat = 29; // Modifiquem  
2 persona.professio = "dissenyadora"; // Afegim nova propietat
```

Mètodes dins d'un objecte literal

A més de propietats amb valors com números o textos, un objecte literal també pot incloure **funcions**. Quan una funció forma part d’un objecte, l’anomenem **mètode**.

Aquests mètodes permeten definir accions o comportaments relacionats amb l'objecte.

```
1 let persona = {  
2   nom: "Anna",  
3   edat: 28,  
4   viu: true,  
5   saludar() {  
6     console.log('Hola! Soc ${this.nom}.');  
7   }  
8 };  
9  
10 persona.saludar(); // "Hola! Soc Anna."
```

Els mètodes poden accedir a les propietats de l'objecte mitjançant la paraula clau `this`, que fa referència a l'objecte mateix.

Eliminar propietats

La paraula clau `delete` serveix per **eliminar propietats d'un objecte**. Quan s'utilitza, **esborra la clau i el valor** associat, de manera que ja no existeixen dins de l'objecte.

```
1 delete persona.viu;
```

També podem eliminar **mètodes** (funcions dins d'un objecte), ja que internament són propietats com qualsevol altra.

```
1 let persona = {  
2   nom: "Anna",  
3   edat: 29,  
4   saludar() {  
5     console.log('Hola, soc ${this.nom}');  
6   }  
7 };  
8  
9 persona.saludar(); // "Hola, soc Anna"  
10 delete persona.saludar; // Eliminem el mètode  
11 console.log(persona.saludar); // undefined
```

Després d'eliminar-lo, el mètode ja no està disponible dins de l'objecte.

Recórrer les propietats amb "for...in"

Quan treballem amb objectes, sovint volem **recórrer totes les seves propietats** per llegir o processar els valors que contenen. En JavaScript, això es pot fer fàcilment amb el bucle especial **for...in**.

Aquest bucle ens permet **iterar per totes les claus (propietats enumerables)** d'un objecte.

```
1 for (let clau in persona) {  
2   console.log(clau + ": " + persona[clau]);  
}
```

```
3 }  
4 // nom: Anna  
5 // edat: 29  
6 // professio: dissenyadora
```

Els objectes són especialment útils quan necessitem **representar entitats complexes** que tenen diverses característiques o propietats. Per exemple, si volem descriure un usuari, un llibre o un producte, cadascun d'aquests elements tindrà múltiples dades associades, com el nom, l'edat, el preu o la categoria. En aquests casos, un array no seria suficient, ja que només ens permet emmagatzemar valors de manera ordenada, però **sense significat semàntic clar**.

A diferència dels arrays, que organitzen la informació per posició (índex), els objectes ens permeten accedir a cada dada mitjançant **una clau descriptiva**, com nom, edat o preu. Això fa que el codi sigui més **llegible i entenedor**, sobretot quan treballem amb dades estructurades.

En resum, utilitzarem objectes sempre que vulguem **estructurar informació amb sentit** (estructures de dades), i no només llistes de valors. Són una eina fonamental per construir programes clars, escalables i fàcils de mantenir.

3.1.3 Estructures de dades complexes

Quan treballem amb dades reals en una aplicació, sovint no n'hi ha prou amb estructures simples com arrays o objectes independents. En aquests casos, JavaScript ens permet **combinar arrays i objectes** per construir **estructures de dades complexes**, molt més potents i representatives del món real.

Arrays que contenen objectes

Una de les combinacions més habituals és tenir un **array d'objectes**. Aquesta estructura és ideal quan volem representar una col·lecció d'elements que comparteixen una mateixa estructura, com ara una llista de llibres, d'usuaris o de productes. Cada objecte de l'array representa un element amb les seves pròpies propietats.

Per exemple, imaginem que volem representar un catàleg de llibres:

```
1 let llibres = [  
2   { titol: "1984", autor: "George Orwell", any: 1949 },  
3   { titol: "Crim i càstig", autor: "Fiódor Dostoievski", any:  
4     1866 },  
5   { titol: "L'ombra del vent", autor: "Carlos Ruiz Zafón", any:  
     2001 }  
];
```

En aquest cas, l'array `llibres` conté tres objectes, cadascun amb informació rellevant sobre un llibre. Això ens permet, per exemple, recórrer tota la llista

amb un bucle i mostrar només els títols o buscar l'autor d'un llibre concret.

Aquesta estructura és especialment útil quan gestionem dades provinents d'una API o una base de dades, ja que aquestes normalment retornen col·leccions d'elements representats com objectes dins d'un array.

Objectes amb propietats que són arrays

També és molt comú el cas invers: **objectes que contenen arrays com a valors d'algunes propietats**. Això ens permet representar característiques que poden tenir **múltiples valors associats**, com per exemple les aficions d'una persona, les notes d'un alumne, o les etiquetes d'un producte.

Observem aquest exemple:

```
1 let usuari = {  
2   nom: "Marta",  
3   edat: 22,  
4   aficions: ["lectura", "escalada", "viatjar"]  
5 };
```

Aquí l'objecte usuari conté una propietat aficions que és un array. Això ens permet treballar de manera eficient amb múltiples valors relacionats amb un sol camp, mantenint una estructura lògica i clara.

Podem accedir-hi i treballar-hi fàcilment:

```
1 console.log(usuari.aficions[1]); // "escalada"  
2 usuari.aficions.push("cinema");
```

Aquest tipus de combinacions ens permeten **modelar la realitat** dins del nostre programa. A diferència d'un full de càlcul o una simple llista de valors, aquí podem representar relacions complexes, jerarquies i col·leccions d'informació estructurada.

És gràcies a aquestes estructures que podem construir funcionalitats com:

- Llistes de tasques amb subtasques
- Perfils d'usuari amb múltiples preferències
- Carretons de compra amb productes i quantitats
- Aplicacions de calendari amb esdeveniments per dies

Dominar les estructures de dades complexes ens permet construir aplicacions **més potents i realistes**. JavaScript ens dona la flexibilitat de **combinar arrays i objectes lliurement**, cosa que és fonamental per treballar amb dades de forma organitzada, neta i escalable.

3.2 Programació orientada a objectes

JavaScript és un llenguatge molt flexible i dinàmic, i una de les seves característiques més potents és que permet programar seguint el paradigma orientat a objectes. Això vol dir que podem organitzar el nostre codi en estructures que encapsulen dades i comportaments, facilitant la reutilització, la modularitat i la comprensió del programa.

Tot i que internament JavaScript es basa en un sistema d'**herència per prototips**, des de la versió **ES6 (2015)** s'introdueixen les **classes**, que ens permeten escriure codi amb una sintaxi més familiar i clara, especialment per a persones acostumades a llenguatges com Java, Python o C++.

La programació orientada a objectes amb JavaScript ens permet **modelar el món real** a través de classes, objectes, propietats i mètodes. Gràcies a la sintaxi moderna de classes, escriure codi estructurat i reutilitzable és molt més senzill i intuïtiu.

Aquest estil de programació és fonamental quan treballem en projectes grans, interfícies complexes o aplicacions que gestionen molts tipus d'entitats i relacions. Aprendre a utilitzar correctament les classes, l'herència i l'encapsulació ens dona una eina potent per escriure **codi net, mantenible i escalable**.

3.2.1 Introducció a les classes

Una classe és un esquema o plantilla que defineix com han de ser els objectes que es creïn a partir d'ella. Ens permet agrupar les propietats i els mètodes que definiran el comportament d'una entitat concreta, com ara un usuari, una mascota, una factura o qualsevol element que volem modelar.

En JavaScript, podem definir una classe utilitzant la paraula clau `class`, i afegir-hi un constructor per inicialitzar les propietats. A diferència d'altres llenguatges, no cal declarar els atributs fora del constructor.

Per exemple, si volem definir una classe `Persona`, podríem escriure el següent:

```
1 class Persona {  
2   constructor(nom, edat) {  
3     this.nom = nom;  
4     this.edat = edat;  
5   }  
6  
7   saludar() {  
8     console.log(`Hola, soc ${this.nom} i tinc ${this.edat} anys`  
9       .');  
10  }  
11 }
```


Quan fem servir el mot clau `new`, estem creant una nova instància d'aquesta classe. Això vol dir que podem crear tantes persones com vulguem, totes amb el seu nom i edat particulars, però compartint el comportament comú definit dins la classe.

```
1 const p1 = new Persona("Anna", 30);
2 const p2 = new Persona("Joan", 22);
3
4 p1.saludar(); // Hola, soc Anna i tinc 30 anys.
5 p2.saludar(); // Hola, soc Joan i tinc 22 anys.
```

3.2.2 El paper de 'this' dins de les classes

Dins d'una classe, la paraula clau `this` fa referència a la instància concreta de l'objecte. És a dir, quan escrivim `this.nom`, estem accedint a la propietat `nom` de l'objecte que s'utilitza en aquell moment. Aquest comportament és essencial per distingir entre els valors que rep el constructor i els que es guarden dins l'objecte.

3.2.3 Encapsulació i modificació de valors

A diferència de llenguatges fortament orientats a objectes, JavaScript no obliga a declarar les propietats com a públiques o privades. Però a partir d'ES2022 ja és possible usar el símbol `#` per definir **propietats privades**, que només poden ser accedides des de dins de la classe.

```
1 class Compte {
2   #saldo;
3
4   constructor(titular) {
5     this.titular = titular;
6     this.#saldo = 0;
7   }
8
9   ingressar(quantitat) {
10    this.#saldo += quantitat;
11  }
12
13  veureSaldo() {
14    return this.#saldo;
15  }
16 }
17
18 const c = new Compte("Laura");
19 c.ingressar(100);
20 console.log(c.veureSaldo()); // 100
21 console.log(c.#saldo);       // Error: propietat privada
```

Aquest sistema ens ajuda a protegir les dades internes i evitar que es modifiquin des de fora de la classe, millorant la seguretat i robustesa del codi.

Propietats públiques vs. privades

JavaScript **obliga** a declarar les propietats privades amb # **a dalt de tot dins de la classe**, abans de fer-les servir dins del constructor o de qualsevol mètode. Això forma part de l'especificació del llenguatge.

Per contra, les propietats **públiques** (com `this.titular`) **no cal declarar-les prèviament**. El fet de fer `this.titular = ...` dins del constructor **ja crea automàticament la propietat a l'objecte**.

Ara bé, sovint volem **controlar encara més l'accés o la modificació de certs valors**, però sense haver d'utilitzar mètodes com `veureSaldo()` o `modificarSaldo()`, que poden resultar menys intuïtius. En aquests casos, JavaScript ens ofereix una eina molt útil: els **accessors de dades** o **data accessors**, que s'implementen mitjançant les paraules clau `get` i `set`.

Els accessors ens permeten definir **com es llegeix (get) o com s'escriu (set) una propietat**, però des del punt de vista extern, l'ús serà igual que si accedíssim directament a una propietat. Això fa que el codi sigui més net i expressiu, i alhora ens permet **validar o transformar les dades en segon pla**.

Per exemple, podem reescriure el cas anterior del `Compte` per fer servir `get` i `set` en lloc de mètodes tradicionals:

```
1 class Compte {
2   #saldo;
3
4   constructor(titular) {
5     this.titular = titular;
6     this.#saldo = 0;
7   }
8
9   get saldo() {
10    return this.#saldo.toFixed(2) + " €";
11  }
12
13  set saldo(quantitat) {
14    if (quantitat >= 0) {
15      this.#saldo = quantitat;
16    } else {
17      console.log("No es pot assignar un saldo negatiu.");
18    }
19  }
20 }
```

Ara, l'ús extern és molt més natural:

```
1 const compte = new Compte("Laura");
2
3 compte.saldo = 200;           // assignació controlada amb el
   set
4 console.log(compte.saldo);    // lectura controlada amb el get
   → "200.00 €"
5
6 compte.saldo = -50;          // no es permet, salta l'avís
```

Amb aquesta estructura, mantenim la protecció de la propietat interna `#saldo`, però oferim **una interfície d'accés elegant i segura**, molt semblant a les propietats públiques d'altres llenguatges orientats a objectes.

Els *data accessors* són especialment útils quan volem que **el comportament d'una propietat sembli directe, però internament amagui lògica de control o format**, i per tant constitueixen una extensió molt natural de l'encapsulació moderna en JavaScript.

3.2.4 Abstracció

L'**abstracció** és un dels pilars fonamentals de la programació orientada a objectes. Consisteix a **ocultar els detalls interns d'una implementació i mostrar només la interfície necessària** per a qui l'utilitza. L'objectiu és **simplificar la complexitat**, evitant que el codi que utilitza un objecte hagi de conèixer com funciona per dins.

En llenguatges com **Java**, l'abstracció es formalitza mitjançant **classes abstractes i interfaces**, que permeten definir què pot fer un objecte sense dir exactament com ho fa. Això facilita la separació entre definició i implementació.

En canvi, JavaScript no té classes abstractes com a construcció formal, però això no vol dir que no pugui implementar l'abstracció. En JavaScript, podem aconseguir l'abstracció encapsulant detalls interns, fent servir propietats privades (amb #) i exposant només mètodes públics per interactuar amb l'objecte.

Exemple d'abstracció en JavaScript:

```
1 class Sensor {  
2   #valorIntern;  
3  
4   constructor() {  
5     this.#valorIntern = Math.random() * 100;  
6   }  
7  
8   obtenirLectura() {  
9     return this.#valorIntern.toFixed(2) + " u.";  
10  }  
11 }
```

En aquest cas, el codi que utilitza el sensor només pot fer:

```
1 const s = new Sensor();  
2 console.log(s.obtenirLectura()); // "47.39 u." (per exemple)
```

No pot accedir a #valorIntern ni modificar-lo directament, perquè forma part de la implementació interna. L'usuari només veu el mètode públic obtenirLectura(), que és tot el que necessita per fer servir correctament l'objecte.

Comparació amb un exemple en Java:

```
1 public abstract class Sensor {  
2   protected double valorIntern;  
3  
4   public Sensor() {  
5     this.valorIntern = Math.random() * 100;  
6   }  
7 }
```

```
7
8     public abstract String obtenirLectura();
9 }
10
11 public class SensorTemperatura extends Sensor {
12     @Override
13     public String obtenirLectura() {
14         return String.format("%.2f ºC", valorIntern);
15     }
16 }
```

I el seu ús seria:

```
1 Sensor s = new SensorTemperatura();
2 System.out.println(s.obtenirLectura());
```

En aquest cas, la **classe abstracta Sensor** defineix **què ha de fer** qualsevol sensor (`obtenirLectura()`), però no com. La classe *filla* (`SensorTemperatura`) implementa el comportament concret. Això és **abstracció formal**, amb suport explícit del llenguatge.

3.2.5 Herència entre classes

La programació orientada a objectes també permet **heretar comportaments** d'una classe base. Això vol dir que podem definir una classe general i després crear altres classes més específiques que parteixen de la primera i afegeixen funcionalitats pròpies.

Per fer-ho, JavaScript ens ofereix les paraules clau `extends` i `super`:

```
1 class Animal {
2     constructor(nom) {
3         this.nom = nom;
4     }
5
6     parlar() {
7         console.log(`${this.nom} fa un so.`);
8     }
9 }
10
11 class Gos extends Animal {
12     parlar() {
13         console.log(`${this.nom} borda.`);
14     }
15 }
16
17 const layca = new Animal("Layca");
18 layca.parlar(); // Layca fa un so.
19 const rex = new Gos("Rex");
20 rex.parlar(); // Rex borda.
```

Quan una classe hereta d'una altra, pot **sobreescriure mètodes** per adaptar-ne el comportament, com hem vist amb el mètode `parlar()`. Aquesta sobreescritura de mètodes és coneix com a **polimorfisme**.

3.2.6 Polimorfisme

El **polimorfisme** ens permet definir mètodes amb el mateix nom però comportaments diferents, segons el tipus d'objecte que els implementa. En JavaScript, això es pot aconseguir mitjançant sobreescritura de mètodes en classes filles.

```
1 class Empleat {
2   saludar() {
3     console.log("Hola, soc un empleat.");
4   }
5 }
6
7 class Cap extends Empleat {
8   saludar() {
9     console.log("Hola, soc el cap!");
10  }
11 }
12
13 const e = new Empleat();
14 const c = new Cap();
15
16 e.saludar(); // "Hola, soc un empleat."
17 c.saludar(); // "Hola, soc el cap!"
```

Tots dos objectes tenen el mètode `saludar()`, però el comportament és diferent: això és **polimorfisme**.

3.2.7 Una menció als prototips

Tot i que actualment la majoria de programadors fan servir la sintaxi de classes, és important saber que **les classes són només una forma més clara de treballar amb els prototips**, que són el veritable sistema d'herència de JavaScript. Cada objecte en JavaScript té un enllaç intern al seu prototip, i si no troba una propietat o mètode en ell mateix, la busca al prototip del qual deriva.

Aquesta cadena d'herència es coneix com a **cadena de prototips**. Tot i això, gràcies a les classes, ja no cal definir manualment mètodes al prototip, cosa que abans es feia així:

```
1 function Persona(nom) {
2   this.nom = nom;
3 }
4
5 Persona.prototype.saludar = function() {
6   console.log('Hola, soc ${this.nom}');
7 };
```

Aquest codi funciona, però és molt menys llegible que la versió moderna amb `class`.

3.3 Exemples pràctics

Després de tota la teoria que hem treballat sobre programació orientada a objectes en JavaScript —incloent l'ús de classes, encapsulació, herència, polimorfisme, abstracció i accessors de dades— és fonamental consolidar aquests coneixements mitjançant exemples pràctics. Això ajuda a visualitzar com aplicar aquests conceptes a situacions reals i fa que entenguem no només la sintaxi, sinó també **l'estructura i la intenció del codi**.

3.3.1 Exemple 1: Estudiants i qualificacions

Es demana crear una petita aplicació per **gestionar les qualificacions d'estudiants** en diferents assignatures.

Cada estudiant té un **nom** i un **DNI** (que ha de ser privat). A més, pot estar matriculat en diverses assignatures, cadascuna amb una **llista de notes**.

Cada assignatura ha de permetre:

- Afegir notes vàlides (entre 0 i 10).
- Calcular automàticament la **mitjana** de les notes amb un accés get.

Cada estudiant ha de poder:

- Afegir assignatures al seu historial.
- Consultar la seva identificació (nom + DNI).
- Mostrar un llistat de totes les assignatures amb la seva mitjana.

Per reforçar la **encapsulació i l'abstracció**, es demana que la informació privada no sigui accessible directament, i que s'ofereixin **mètodes públics** per interactuar amb l'objecte de forma segura.

```
1 class Assignatura {
2   constructor(nom) {
3     this.nom = nom;
4     this.notes = [];
5   }
6
7   afegirNota(nota) {
8     if (nota >= 0 && nota <= 10) {
9       this.notes.push(nota);
10    } else {
11      console.log("Nota no vàlida");
12    }
13  }
```

```
14
15   get mitjana() {
16     const total = this.notes.reduce((acc, n) => acc + n, 0);
17     return this.notes.length ? (total / this.notes.length).
       toFixed(2) : "No hi ha notes";
18   }
19 }
20
21 class Estudiant {
22   #dni; // propietat privada
23
24   constructor(nom, dni) {
25     this.nom = nom;
26     this.#dni = dni;
27     this.assignatures = [];
28   }
29
30   get identificador() {
31     return `Estudiant: ${this.nom} (DNI: ${this.#dni})`;
32   }
33
34   afegirAssignatura(assignatura) {
35     this.assignatures.push(assignatura);
36   }
37
38   mostrarQualificacions() {
39     console.log(this.identificador);
40     this.assignatures.forEach(a => {
41       console.log(`- ${a.nom}: mitjana = ${a.mitjana}`);
42     });
43   }
44 }
45
46 // Ús
47 const anna = new Estudiant("Anna Martí", "12345678A");
48 const mates = new Assignatura("Matemàtiques");
49 const catala = new Assignatura("Català");
50
51 mates.afegirNota(7.5);
52 mates.afegirNota(9);
53 catala.afegirNota(8.2);
54
55 anna.afegirAssignatura(mates);
56 anna.afegirAssignatura(catala);
57
58 anna.mostrarQualificacions();
59   // Estudiant: Anna Martí (DNI: 12345678A)
60   // - Matemàtiques: mitjana = 8.25
61   // - Català: mitjana = 8.20
```

Aquest exemple mostra clarament l'**encapsulació de dades sensibles (DNI)**, l'**ús de get per obtenir la mitjana** de notes i com l'objecte Estudiant **agrega altres objectes** (Assignatura). A més, la funcionalitat `mostrarQualificacions()` actua com a **interfície pública**, amagant la complexitat interna.

3.3.2 Exemple 2: Jerarquia d'animals

Es vol programar un sistema per gestionar diferents animals d'un petit zoològic. Cada animal té un nom i és capaç d'emetre un so, però **el so canvia segons l'espècie**.

Es demana:

- Crear una **classe base Animal** amb un mètode `ferSo()` genèric.
- Crear subclasses com `Gos`, `Gat` i `Ocell`, que **hereten d'Animal** i **sobreescrueixen el mètode `ferSo()`** per adaptar-lo a l'animal.
- Instanciar diversos animals amb noms propis.
- Emmagatzemar-los en un array.
- Recórrer l'array i cridar el mètode `ferSo()` de cadascun.

Aquest exercici ha de servir per entendre com funciona l'**herència**, el **polimorfisme** i l'ús de **`super()`** i **`this`** dins de la jerarquia d'objectes.

```
1 class Animal {
2   constructor(nom) {
3     this.nom = nom;
4   }
5
6   ferSo() {
7     console.log(`${this.nom} fa un so indefinit.`);
8   }
9 }
10
11 class Gos extends Animal {
12   ferSo() {
13     console.log(`${this.nom} borda: guau!`);
14   }
15 }
16
17 class Gat extends Animal {
18   ferSo() {
19     console.log(`${this.nom} miola: mèu!`);
20   }
21 }
22
23 class Ocell extends Animal {
24   ferSo() {
25     console.log(`${this.nom} canta: piu piu!`);
26   }
27 }
28
29 // Ús
30 const rex = new Gos("Rex");
31 const luna = new Gat("Luna");
32 const kiwi = new Ocell("Kiwi");
33
34 const animals = [rex, luna, kiwi];
```



```
35  
36 animals.forEach(a => a.ferSo());  
37 // Rex borda: guau!  
38 // Luna miola: mèu!  
39 // Kiwi canta: piu piu!
```

Aquest exemple mostra com diferents subclasses poden sobreescriure el mateix mètode (`ferSo`) per oferir comportaments diferents segons l'espècie, sense necessitat de modificar la classe base. És un cas clar de **polimorfisme en acció**.